



MCA SEM-I

SUBJECT: Open-Source Web Programming Using PHP

UNIT- I

What is PHP?

PHP is an open-source, interpreted, and object-oriented scripting language that can be executed at the server-side. PHP is well suited for web development. Therefore, it is used to develop web applications (an application that executes on the server and generates the dynamic page.).

PHP was created by **Rasmus Lerdorf** in **1994** but appeared in the market in 1995. **PHP 7.4.0** is the latest version of PHP, which was released on **28 November**. Some important points need to be noticed about PHP are as followed:

- PHP stands for Hypertext Preprocessor.
- PHP is an interpreted language, i.e., there is no need for compilation.
- PHP is faster than other scripting languages, for example, ASP and JSP.
- PHP is a server-side scripting language, which is used to manage the dynamic content of the website.
- PHP can be embedded into HTML.
- PHP is an object-oriented language.
- PHP is an open-source scripting language.
- PHP is simple and easy to learn language.





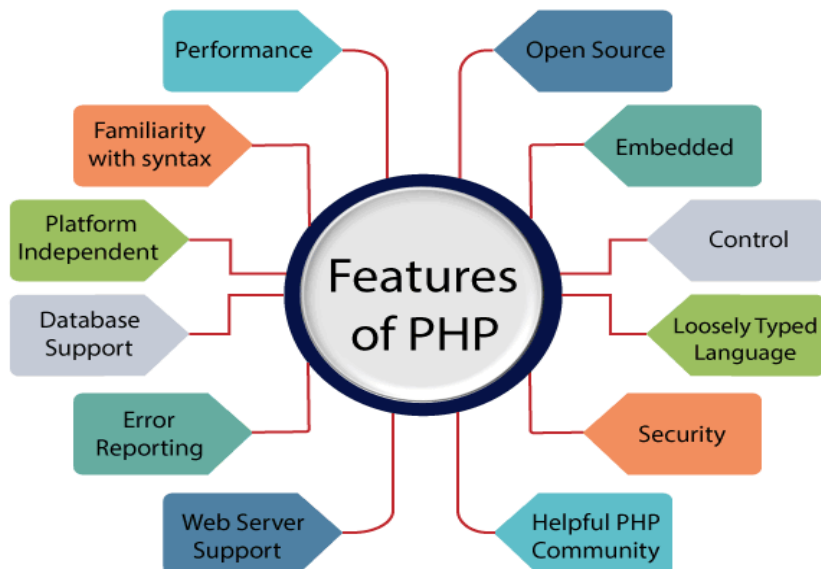
Why use PHP

PHP is a server-side scripting language, which is used to design the dynamic web applications with MySQL database.

- It handles dynamic content, database as well as session tracking for the website.
- You can create sessions in PHP.
- It can access cookies variable and also set cookies.
- It helps to encrypt the data and apply validation.
- PHP supports several protocols such as HTTP, POP3, SNMP, LDAP, IMAP, and many more.
- Using PHP language, you can control the user to access some pages of your website.
- As PHP is easy to install and set up, this is the main reason why PHP is the best language to learn.
- PHP can handle the forms, such as - collect the data from users using forms, save it into the database, and return useful information to the user. **For example** - Registration form.

PHP Features

PHP is very popular language because of its simplicity and open source. There are some important features of PHP given below:





Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

Performance:

PHP script is executed much faster than those scripts which are written in other languages such as JSP and ASP. PHP uses its own memory, so the server workload and loading time is automatically reduced, which results in faster processing speed and better performance.

Open Source:

PHP source code and software are freely available on the web. You can develop all the versions of PHP according to your requirement without paying any cost. All its components are free to download and use.

Familiarity with syntax:

PHP has easily understandable syntax. Programmers are comfortable coding with it.

Embedded:

PHP code can be easily embedded within HTML tags and script.

Platform Independent:

PHP is available for WINDOWS, MAC, and LINUX & UNIX operating system. A PHP application developed in one OS can be easily executed in other OS also.

Database Support:

PHP supports all the leading databases such as MySQL, SQLite, ODBC, etc.

Error Reporting -

PHP has predefined error reporting constants to generate an error notice or warning at runtime. E.g., E_ERROR, E_WARNING, E_STRICT, E_PARSE.

Loosely Typed Language:

PHP allows us to use a variable without declaring its datatype. It will be taken automatically at the time of execution based on the type of data it contains on its value.

Web servers Support:

PHP is compatible with almost all local servers used today like Apache, Netscape, Microsoft IIS, etc.

Security:



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra
Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

PHP is a secure language to develop the website. It consists of multiple layers of security to prevent threats and malicious attacks.

Control:

Different programming languages require long script or code, whereas PHP can do the same work in a few lines of code. It has maximum control over the websites like you can make changes easily whenever you want.

A Helpful PHP Community:

It has a large community of developers who regularly updates documentation, tutorials, online help, and FAQs. Learning PHP from the communities is one of the significant benefits.

Web Development

PHP is widely used in web development nowadays. PHP can develop dynamic websites easily. But you must have the basic the knowledge of following technologies for web development as well.

- HTML
- CSS
- JavaScript
- XML and JSON
-

Install PHP

To install PHP, we will suggest you to install AMP (Apache, MySQL, PHP) software stack. It is available for all operating systems. There are many AMP options available in the market that are given below:

- **WAMP** for Windows
- **LAMP** for Linux
- **MAMP** for Mac
- **SAMP** for Solaris
- **FAMP** for FreeBSD
- **XAMPP** (Cross, Apache, MySQL, PHP, Perl) for Cross Platform: It includes some other components too such as FileZilla, OpenSSL, Webalizer, Mercury Mail, etc.

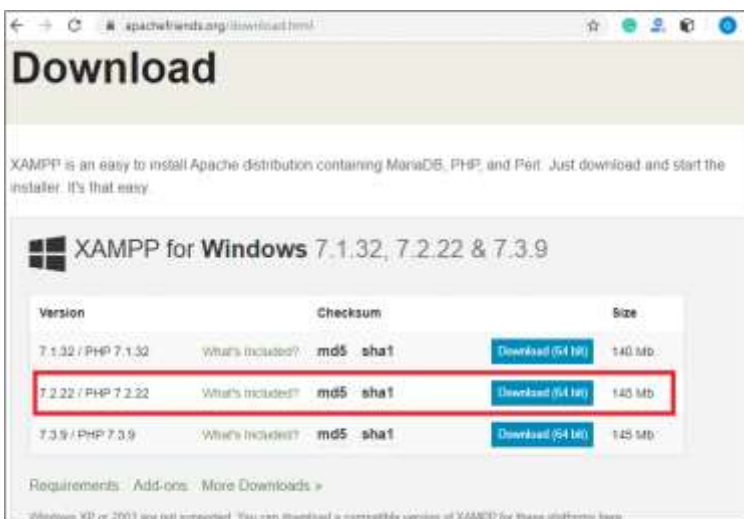


If you are on Windows and don't want Perl and other features of XAMPP, you should go for WAMP. In a similar way, you may use LAMP for Linux and MAMP for Macintosh.

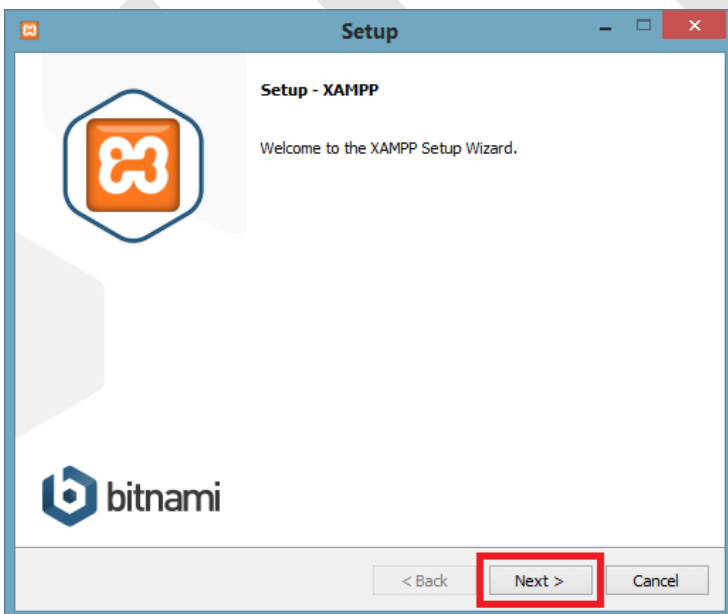
How to install XAMPP server on windows

We will learn how to install the XAMPP server on windows platform step by step. Follow the below steps and install the XAMPP server on your system.

Step 1: Click on the above link provided to download the **XAMPP server** according to your window requirement.



Step 2: After downloading XAMPP, double click on the downloaded file and allow XAMPP to make changes in your system. A window will pop-up, where you have to click on the **Next** button.



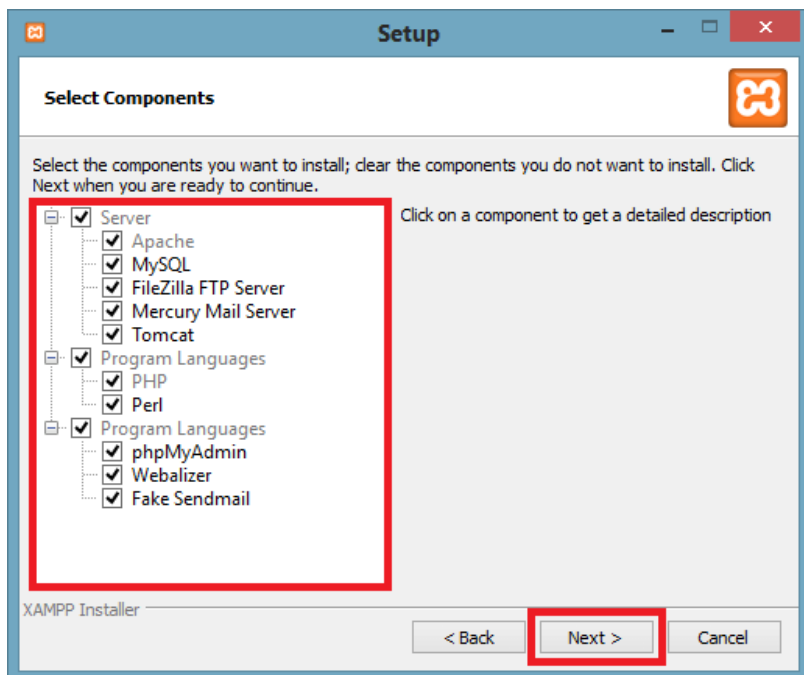


Insight Institute of Technology and Management, Nagpur

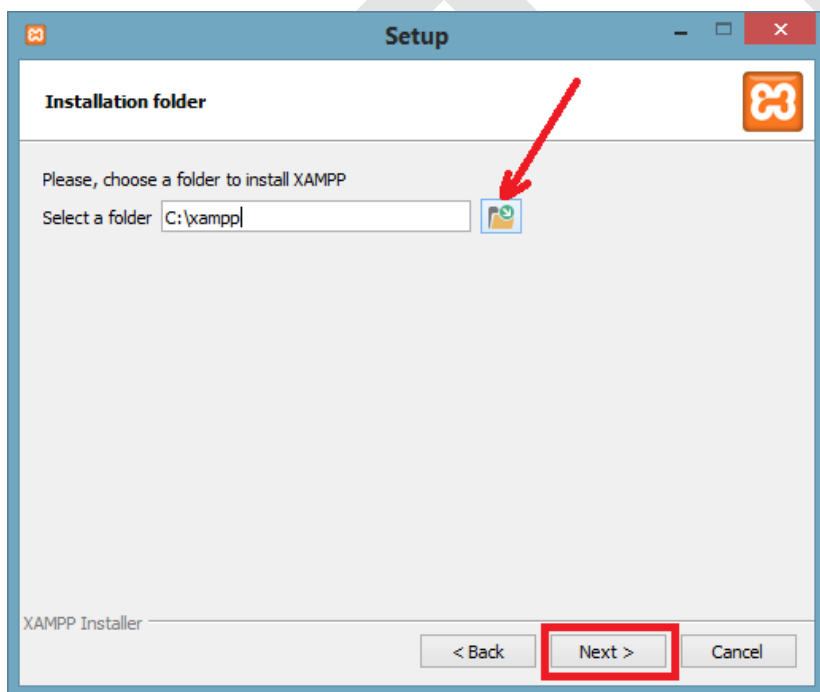
Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

Step 3: Here, select the components, which you want to install and click **Next**.



Step 4: Choose a folder where you want to install the XAMPP in your system and click **next**.

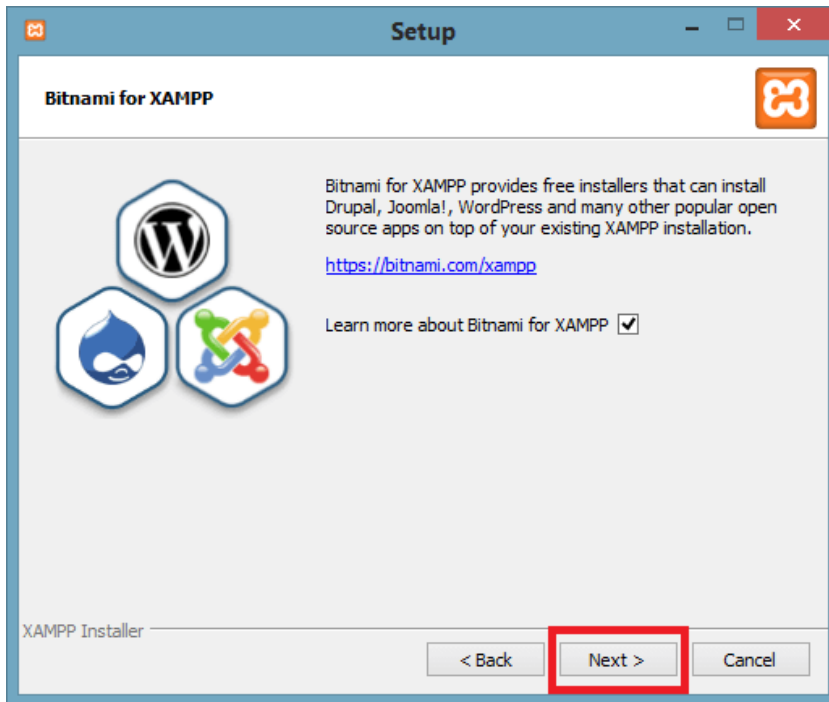




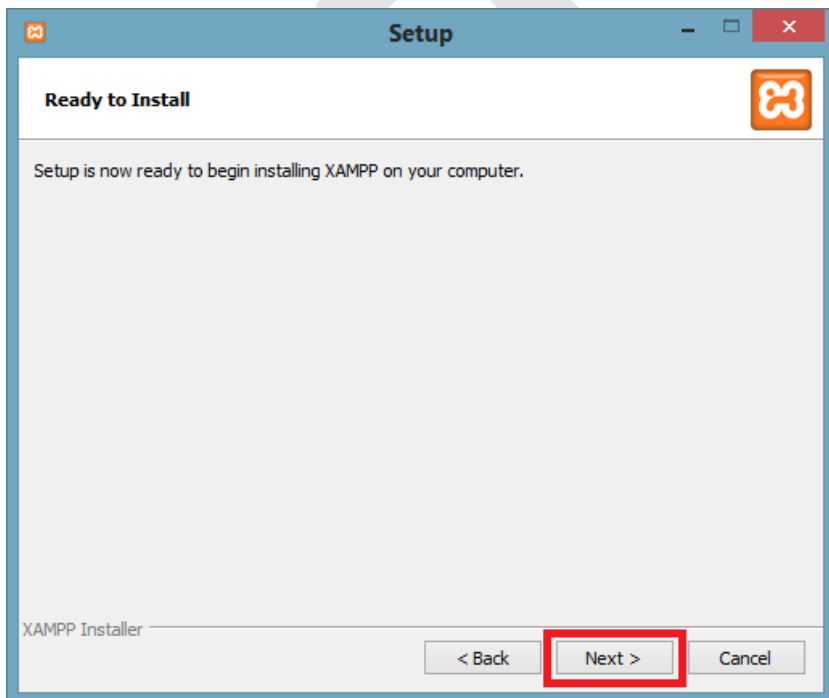
Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra
Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

Step 5: Click **Next** and move ahead.



Step 6: XAMPP is ready to install, so click on the **Next** button and install the XAMPP.



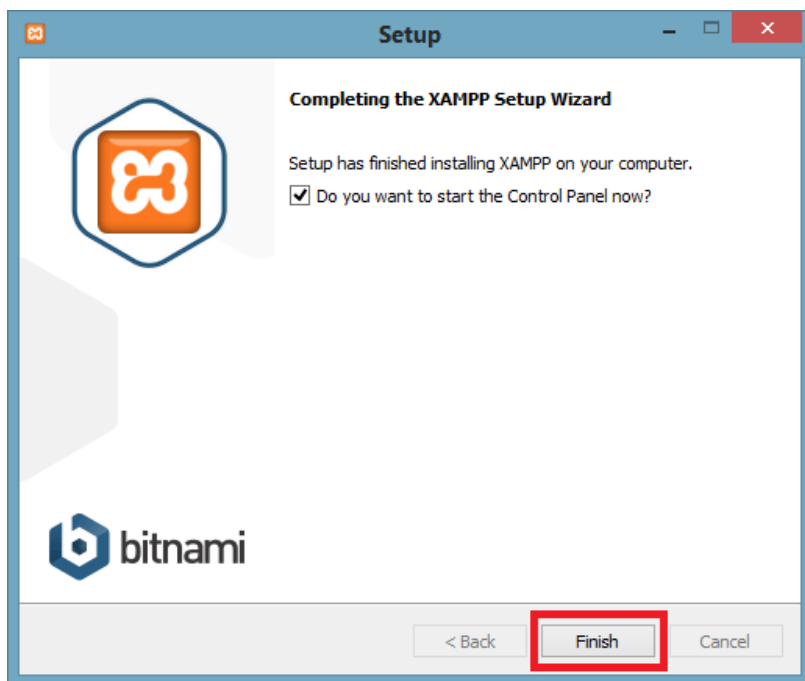


Insight Institute of Technology and Management, Nagpur

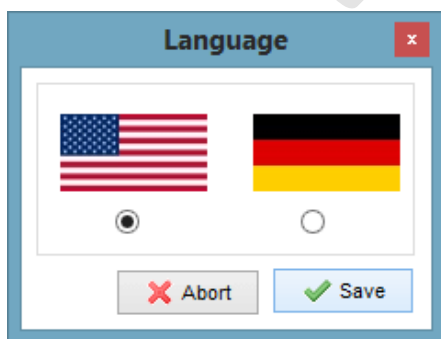
Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

Step 7: A finish window will display after successful installation. Click on the **Finish** button.



Step 8: Choose your preferred language.

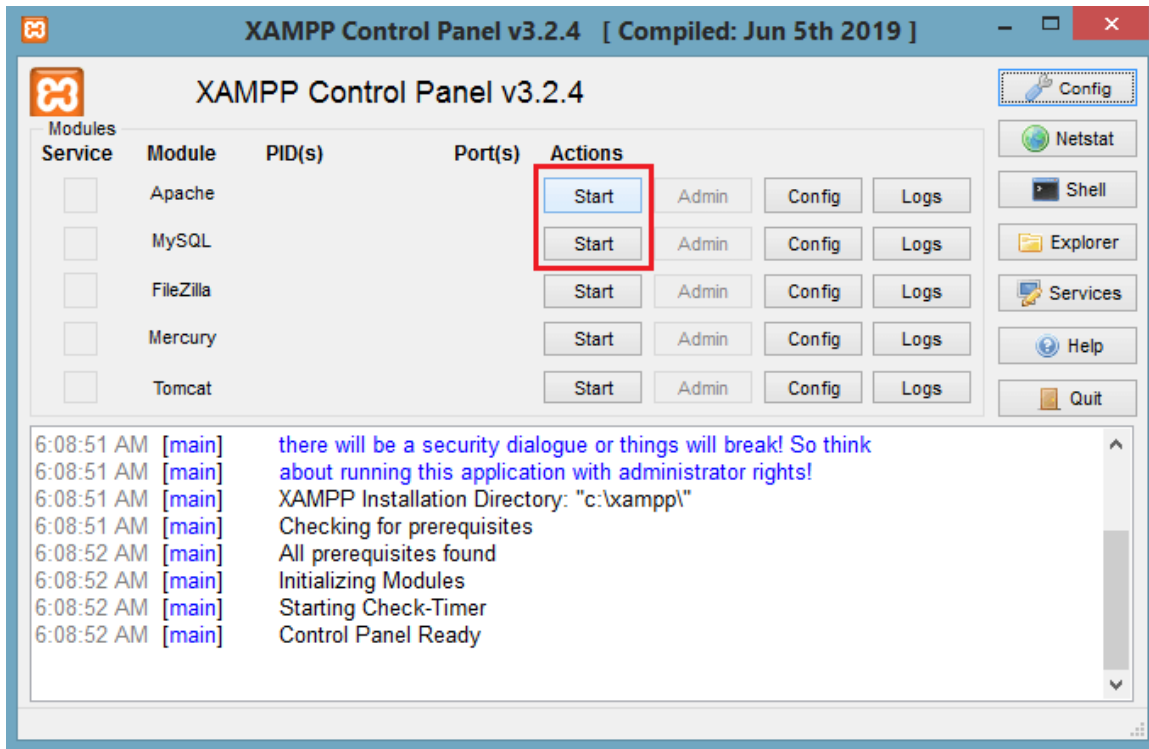




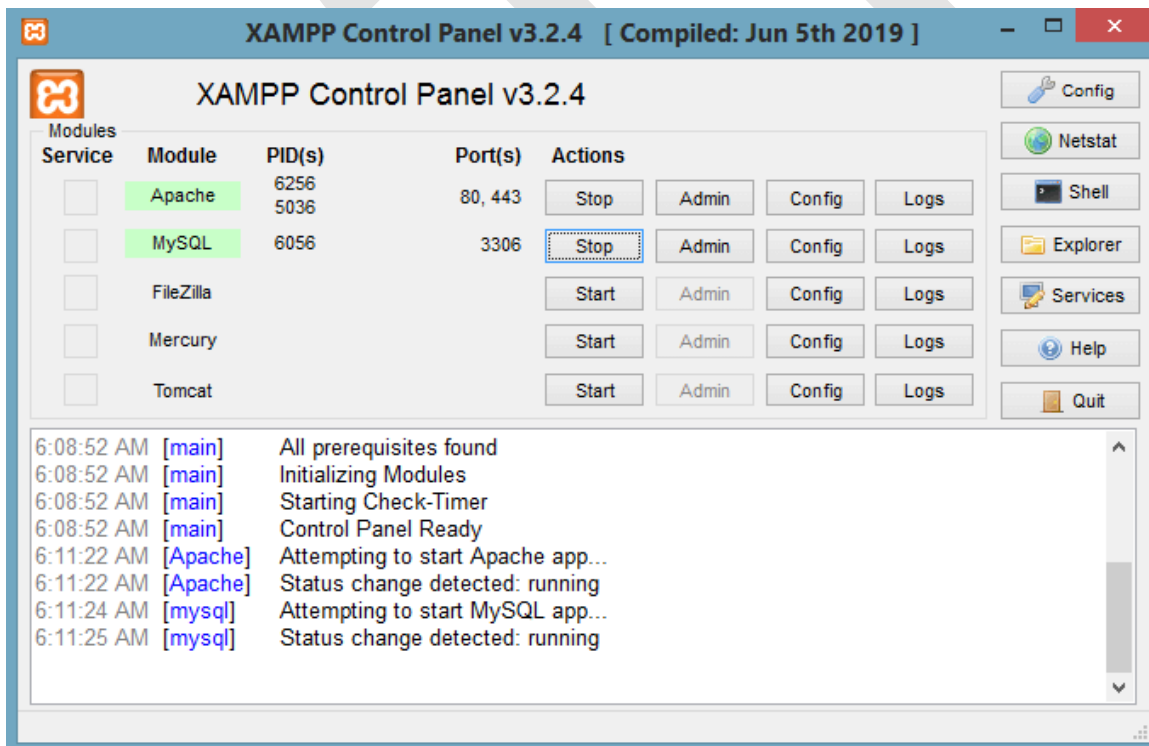
Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra
Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

Step 9: XAMPP is ready to use. Start the Apache server and MySQL and run the php program on the localhost.



Step 10: If no error is shown, then XAMPP is running successfully.





Lexical Structures in PHP :

Lexical structure of a programming language is the set of basic rules that governs how you write programs in that language. The lowest-level syntax of the language and specifies such things as what variable names look like, what characters are used for comments, and how program statements are separated from each other.

Generally, a PHP file contains HTML tags and some PHP scripting code. It is very easy to create a simple PHP example. To do so, create a file and write HTML tags + PHP code and save this file with .php extension.

All PHP code goes between the php tag. It starts with `<?php` and ends with `?>`. The syntax of PHP tag is given below:

1. `<?php`
2. `//your code here`
3. `?>`

Let's see a simple PHP example where we are writing some text using PHP echo command.

1. `<!DOCTYPE>`
2. `<html>`
3. `<body>`
4. `<?php`
5. `echo "<h2>Hello First PHP</h2>";`
6. `?>`
7. `</body>`
8. `</html>`

Output:

Hello First PHP

How to run PHP programs in XAMPP

How to run PHP programs in XAMPP PHP is a popular backend programming language. PHP programs can be written on any editor, such as - Notepad, Notepad++, Dreamweaver, etc. These programs save with .php extension, i.e., filename.php inside the htdocs folder.



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra
Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

For example - p1.php.

As I'm using window, and my XAMPP server is installed in D drive. So, the path for the htdocs directory will be "D:\xampp\htdocs".

PHP program runs on a web browser such as - Chrome, Internet Explorer, Firefox, etc. Below some steps are given to run the PHP programs.

Step 1: Create a simple PHP program like hello world.

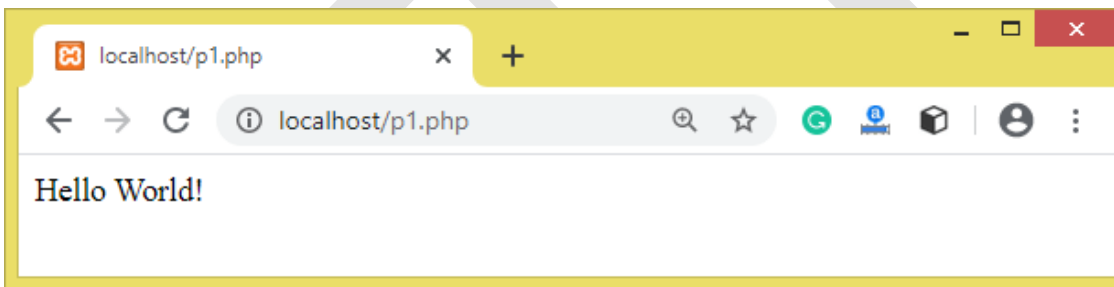
1. `<?php`
2. `echo "Hello World!";`
3. `?>`

Step 2: Save the file with **hello.php** name in the htdocs folder, which resides inside the xampp folder.

Step 3: Run the XAMPP server and start the Apache and MySQL.

Step 4: Now, open the web browser and type localhost *http://localhost/hello.php* on your browser window.

Step 5: The output for the above **hello.php** program will be shown as the screenshot below:



PHP Case Sensitivity

In PHP, keyword (e.g., echo, if, else, while), functions, user-defined functions, classes are not case-sensitive. However, all variable names are case-sensitive.

In the below example, you can see that all three echo statements are equal and valid:

1. `<!DOCTYPE>`
2. `<html>`
3. `<body>`
4. `<?php`
5. `echo "Hello world using echo </br>";`



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

6. ECHO "Hello world using ECHO </br>";
7. EcHo "Hello world using EcHo </br>";
8. ?>
9. </body>
10. </html>

Output:

```
Hello world using echo
Hello world using ECHO
Hello world using EcHo
```

Look at the below example that the variable names are case sensitive. You can see the example below that only the second statement will display the value of the \$color variable. Because it treats \$color, \$ColoR, and \$COLOR as three different variables:

1. <html>
2. <body>
3. <?php
4. \$color = "black";
5. echo "My car is ". \$ColoR . "</br>";
6. echo "My dog is ". \$color . "</br>";
7. echo "My Phone is ". \$COLOR . "</br>";
8. ?>
9. </body>
10. </html>

Output:

Notice: Undefined variable: ColoR in D:\xampp\htdocs\program\p2.php on line 8

```
My car is
My dog is black
```

Notice: Undefined variable: COLOR in D:\xampp\htdocs\program\p2.php on line 10

```
My Phone is
```



PHP Echo

PHP echo is a language construct, not a function. Therefore, you don't need to use parenthesis with it. But if you want to use more than one parameter, it is required to use parenthesis.

The syntax of PHP echo is given below:

1. `void echo ( string $arg1 [, string $... ] )`

PHP echo statement can be used to print the string, multi-line strings, escaping characters, variable, array, etc. Some important points that you must know about the echo statement are:

- echo is a statement, which is used to display the output.
- echo can be used with or without parentheses: `echo()`, and `echo`.
- echo does not return any value.
- We can pass multiple strings separated by a comma (,) in echo.
- echo is faster than the print statement.

PHP echo: printing string

1. `<?php`
2. `echo "Hello by PHP echo";`
3. `?>`

Output:

```
Hello by PHP echo
```

PHP Variables

In PHP, a variable is declared using a **\$ sign** followed by the variable name. Here, some important points to know about variables:

- As PHP is a loosely typed language, so we do not need to declare the data types of the variables. It automatically analyzes the values and makes conversions to its correct datatype.
- After declaring a variable, it can be reused throughout the code.
- Assignment Operator (`=`) is used to assign the value to a variable.



Syntax of declaring a variable in PHP is given below:

1. `$variablename=value;`

Rules for declaring PHP variable:

- A variable must start with a dollar (\$) sign, followed by the variable name.
- It can only contain alpha-numeric character and underscore (A-z, 0-9, _).
- A variable name must start with a letter or underscore (_) character.
- A PHP variable name cannot contain spaces.
- One thing to be kept in mind that the variable name cannot start with a number or special symbols.
- PHP variables are case-sensitive, so \$name and \$NAME both are treated as different variable.

PHP Variable: Declaring string, integer, and float

Let's see the example to store string, integer, and float values in PHP variables.

1. `<?php`
2. `$str="hello string";`
3. `$x=200;`
4. `$y=44.6;`
5. `echo "string is: $str <br/>";`
6. `echo "integer is: $x <br/>";`
7. `echo "float is: $y <br/>";`
8. `?>`

Output:

```
string is: hello string
integer is: 200
float is: 44.6
```

PHP Variable: Sum of two variables

1. `<?php`
2. `$x=5;`
3. `$y=6;`



4. `$z=$x+$y;`
5. `echo $z;`
6. `?>`

Output:

```
11
```

PHP Variable: case sensitive

In PHP, variable names are case sensitive. So variable name "color" is different from Color, COLOR, COlOr etc.

1. `<?php`
2. `$color="red";`
3. `echo "My car is " . $color . "<br>;`
4. `echo "My house is " . $COLOR . "<br>;`
5. `echo "My boat is " . $coLOR . "<br>;`
6. `?>`

Output:

```
My car is red
Notice: Undefined variable: COLOR in C:\wamp\www\variable.php on line 4
My house is
Notice: Undefined variable: coLOR in C:\wamp\www\variable.php on line 5
My boat is
```

PHP \$ and \$\$ Variables

The **\$var** (single dollar) is a normal variable with the name var that stores any value like string, integer, float, etc.

The **\$\$var** (double dollar) is a reference variable that stores the value of the \$variable inside it.

To understand the difference better, let's see some examples.

Example 1

1. `<?php`
2. `$x = "abc";`
3. `$$x = 200;`



4. `echo $x."<br/>";`
5. `echo $$x."<br/>";`
6. `echo $abc;`
7. `?>`

Output:



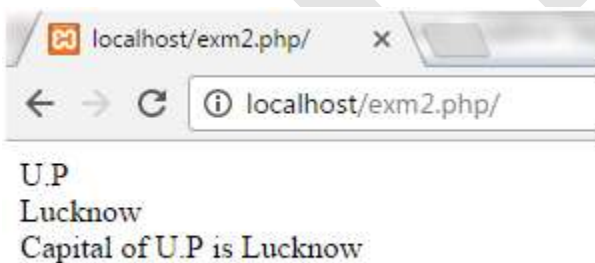
In the above example, we have assigned a value to the variable **x** as **abc**. Value of reference variable **\$\$x** is assigned as **200**.

Now we have printed the values **\$x**, **\$\$x** and **\$abc**.

Example2

1. `<?php`
2. `$x="U.P";`
3. `$$x="Lucknow";`
4. `echo $x. "<br>";`
5. `echo $$x. "<br>";`
6. `echo "Capital of $x is " . $$x;`
7. `?>`

Output:





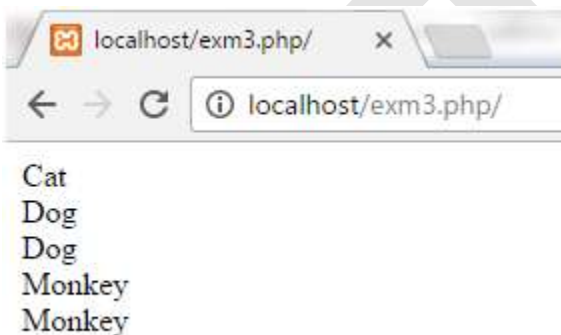
In the above example, we have assigned a value to the variable **x** as **U.P.** Value of reference variable **\$\$x** is assigned as **Lucknow**.

Now we have printed the values **\$x**, **\$\$x** and a string.

Example3

1. <?php
2. \$name="Cat";
3. \${\$name}="Dog";
4. \${\${\$name}}="Monkey";
5. echo \$name. "
";
6. echo \${\$name}. "
";
7. echo \$Cat. "
";
8. echo \${\${\$name}}. "
";
9. echo \$Dog. "
";
10. ?>

Output:



In the above example, we have assigned a value to the variable name **Cat**. Value of reference variable **\${\$name}** is assigned as **Dog** and **\${\${\$name}}** as **Monkey**.

Now we have printed the values as **\$name**, **\${\$name}**, **\$Cat**, **\${\${\$name}}** and **\$Dog**.



PHP Data Types

PHP data types are used to hold different types of data or values. PHP supports 8 primitive data types that can be categorized further in 3 types:

1. Scalar Types (predefined)
2. Compound Types (user-defined)
3. Special Types

PHP Data Types: Scalar Types

It holds only single value. There are 4 scalar data types in PHP.

1. [boolean](#)
2. [integer](#)
3. [float](#)
4. [string](#)

PHP Data Types: Compound Types

It can hold multiple values. There are 2 compound data types in PHP.

1. [array](#)
2. [object](#)

PHP Data Types: Special Types

There are 2 special data types in PHP.

1. [resource](#)
2. [NULL](#)

PHP Boolean

Booleans are the simplest data type works like switch. It holds only two values: **TRUE (1)** or **FALSE (0)**. It is often used with conditional statements. If the condition is correct, it returns TRUE otherwise FALSE.

Example:



```
1. <?php
2.     if (TRUE)
3.         echo "This condition is TRUE.";
4.     if (FALSE)
5.         echo "This condition is FALSE.";
6. ?>
```

Output:

```
This condition is TRUE.
```

PHP Integer

Integer means numeric data with a negative or positive sign. It holds only whole numbers, i.e., numbers without fractional part or decimal points.

Rules for integer:

- An integer can be either positive or negative.
- An integer must not contain decimal point.
- Integer can be decimal (base 10), octal (base 8), or hexadecimal (base 16).
- The range of an integer must be lie between 2,147,483,648 and 2,147,483,647 i.e., -2^{31} to 2^{31} .

Example:

```
1. <?php
2.     $dec1 = 34;
3.     $oct1 = 0243;
4.     $hexa1 = 0x45;
5.     echo "Decimal number: " . $dec1 . "</br>";
6.     echo "Octal number: " . $oct1 . "</br>";
7.     echo "HexaDecimal number: " . $hexa1 . "</br>";
8. ?>
```

Output:

```
Decimal number: 34
Octal number: 163
```



HexaDecimal number: 69

PHP Float

A floating-point number is a number with a decimal point. Unlike integer, it can hold numbers with a fractional or decimal point, including a negative or positive sign.

Example:

```
1. <?php
2.     $n1 = 19.34;
3.     $n2 = 54.472;
4.     $sum = $n1 + $n2;
5.     echo "Addition of floating numbers: " . $sum;
6. ?>
```

Output:

Addition of floating numbers: 73.812

PHP String

A string is a non-numeric data type. It holds letters or any alphabets, numbers, and even special characters.

String values must be enclosed either within **single quotes** or in **double quotes**. But both are treated differently. To clarify this, see the example below:

Example:

```
1. <?php
2.     $company = "Javatpoint";
3.     //both single and double quote statements will treat different
4.     echo "Hello $company";
5.     echo "</br>";
6.     echo 'Hello $company';
7. ?>
```

Output:

Hello Javatpoint
Hello \$company



PHP Array

An array is a compound data type. It can store multiple values of same data type in a single variable.

Example:

```
1. <?php
2.   $bikes = array ("Royal Enfield", "Yamaha", "KTM");
3.   var_dump($bikes); //the var_dump() function returns the datatype and values
4.   echo "</br>";
5.   echo "Array Element1: $bikes[0] </br>";
6.   echo "Array Element2: $bikes[1] </br>";
7.   echo "Array Element3: $bikes[2] </br>";
8. ?>
```

Output:

```
array(3) { [0]=> string(13) "Royal Enfield" [1]=> string(6) "Yamaha" [2]=> string(3) "KTM" }
Array Element1: Royal Enfield
Array Element2: Yamaha
Array Element3: KTM
```

You will learn more about array in later chapters of this tutorial.

PHP object

Objects are the instances of user-defined classes that can store both values and functions. They must be explicitly declared.

Example:

```
1. <?php
2.   class bike {
3.       function model() {
4.           $model_name = "Royal Enfield";
5.           echo "Bike Model: " . $model_name;
6.       }
7.   }
8.   $obj = new bike();
```



9. \$obj -> model();
10. ?>

Output:

Bike Model: Royal Enfield

This is an advanced topic of PHP, which we will discuss later in detail.

PHP Resource

Resources are not the exact data type in PHP. Basically, these are used to store some function calls or references to external PHP resources. **For example** - a database call. It is an external resource.

This is an advanced topic of PHP, so we will discuss it later in detail with examples.

PHP Null

Null is a special data type that has only one value: **NULL**. There is a convention of writing it in capital letters as it is case sensitive.

The special type of data type NULL defined a variable with no value.

Example:

1. <?php
2. \$nl = NULL;
3. echo \$nl; *//it will not give any output*
4. ?>



PHP Operators

PHP also supports various types of operations like arithmetic operations(addition, subtraction, etc), logical operations(AND, OR etc), Increment/Decrement Operations, etc. Thus, PHP provides us with many operators to perform such operations on various operands or variables, or values. These operators are nothing but symbols needed to perform operations of various types. Given below are the various groups of operators:

- Arithmetic Operators
- Logical or Relational Operators
- Comparison Operators
- Conditional or Ternary Operators
- Assignment Operators
- Spaceship Operators (Introduced in PHP 7)
- Array Operators
- Increment/Decrement Operators
- String Operators

Let us now learn about each of these operators in detail.

Arithmetic Operators:

The arithmetic operators are used to perform simple mathematical operations like addition, subtraction, multiplication, etc. Below is the list of arithmetic operators along with their syntax and operations in PHP.

Operator	Name	Syntax	Operation
+	Addition	$\$x + \y	Sum the operands
-	Subtraction	$\$x - \y	Differences the operands
*	Multiplication	$\$x * \y	Product of the operands
/	Division	$\$x / \y	The quotient of the operands
**	Exponentiation	$\$x ** \y	$\$x$ raised to the power $\$y$
%	Modulus	$\$x \% \y	The remainder of the operands

- PHP



```
<?php
```

```
$x = 29; // Variable 1
```

```
$y = 4; // Variable 2
```

```
// Some arithmetic operations on
```

```
// these two variables
```

```
echo ($x + $y), "\n";
```

```
echo($x - $y), "\n";
```

```
echo($x * $y), "\n";
```

```
echo($x / $y), "\n";
```

```
echo($x % $y), "\n";
```

```
?>
```

Output:

33

25

116

7.25

1

Logical or Relational Operators:

These are basically used to operate with conditional statements and expressions. Conditional statements are based on conditions. Also, a condition can either be met or cannot be met so the result of a conditional statement can either be true or false. Here are the logical operators along with their syntax and operations in PHP.



Operator	Name	Syntax	Operation
and	Logical AND	\$x and \$y	True if both the operands are true else false
or	Logical OR	\$x or \$y	True if either of the operands is true else false
xor	Logical XOR	\$x xor \$y	True if either of the operands is true and false if both are true
&&	Logical AND	\$x && \$y	True if both the operands are true else false
	Logical OR	\$x \$y	True if either of the operands is true else false
!	Logical NOT	!\$x	True if \$x is false

Example: This example describes the logical & relational operator in PHP.

- PHP

```
<?php
```

```
$x = 50;
```

```
$y = 30;
```

```
if ($x == 50 and $y == 30)
```

```
    echo "and Success \n";
```

```
if ($x == 50 or $y == 20)
```

```
    echo "or Success \n";
```

```
if ($x == 50 xor $y == 20)
```



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

```
echo "xor Success \n";
```

```
if ($x == 50 && $y == 30)
```

```
echo "&& Success \n";
```

```
if ($x == 50 || $y == 20)
```

```
echo "|| Success \n";
```

```
if (!$z)
```

```
echo "! Success \n";
```

```
?>
```

Output:

and Success

or Success

xor Success

&& Success

|| Success

! Success

Comparison Operators: These operators are used to compare two elements and outputs the result in boolean form. Here are the comparison operators along with their syntax and operations in PHP.

Operator	Name	Syntax	Operation
==	Equal To	$\$x == \y	Returns True if both the operands are equal
!=	Not Equal To	$\$x != \y	Returns True if both the operands are not equal
<>	Not Equal To	$\$x <> \y	Returns True if both the operands are unequal



Operator	Name	Syntax	Operation
===	Identical	$\$x === \y	Returns True if both the operands are equal and are of the same type
!==	Not Identical	$\$x !== \y	Returns True if both the operands are unequal and are of different types
<	Less Than	$\$x < \y	Returns True if \$x is less than \$y
>	Greater Than	$\$x > \y	Returns True if \$x is greater than \$y
<=	Less Than or Equal To	$\$x \leq \y	Returns True if \$x is less than or equal to \$y
>=	Greater Than or Equal To	$\$x \geq \y	Returns True if \$x is greater than or equal to \$y

Example: This example describes the comparison operator in PHP.

- PHP

```
<?php
```

```
$a = 80;
```

```
$b = 50;
```

```
$c = "80";
```

```
// Here var_dump function has been used to
```



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

// display structured information. We will learn

// about this function in complete details in further

// articles.

```
var_dump($a == $c) + "\n";
```

```
var_dump($a != $b) + "\n";
```

```
var_dump($a <> $b) + "\n";
```

```
var_dump($a === $c) + "\n";
```

```
var_dump($a !== $c) + "\n";
```

```
var_dump($a < $b) + "\n";
```

```
var_dump($a > $b) + "\n";
```

```
var_dump($a <= $b) + "\n";
```

```
var_dump($a >= $b);
```

```
?>
```

Output:

```
bool(true)
```

```
bool(true)
```

```
bool(true)
```

```
bool(false)
```

```
bool(true)
```

```
bool(false)
```

```
bool(true)
```

```
bool(false)
```

Prof. SHREYA BHANSE

Open-Source Web Programming using PHP



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra
Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

bool(true)

Conditional or Ternary Operators:

These operators are used to compare two values and take either of the results simultaneously, depending on whether the outcome is TRUE or FALSE. These are also used as a shorthand notation for *if...else* statement that we will read in the article on decision making.

Syntax:

`$var = (condition)? value1 : value2;`

Here, the condition will either evaluate as true or false. If the condition evaluates to True, then value1 will be assigned to the variable \$var otherwise value2 will be assigned to it.

Operator	Name	Operation
?:	Ternary	If the condition is true? Then \$x: or else \$y. This means that if the condition is true then the left result of the colon is accepted otherwise the result is on right.

Example: This example describes the Conditional or Ternary operators in PHP.

- PHP

```
<?php
```

```
$x = -12;
```

```
echo ($x > 0) ? 'The number is positive' : 'The number is negative';
```

```
?>
```

Output:

The number is negative

Assignment Operators: These operators are used to assign values to different variables, with or without mid-operations. Here are the assignment operators along with their syntax and operations, that PHP provides for the operations.

Operator	Name	Syntax	Operation
=	Assign	<code>\$x = \$y</code>	Operand on the left obtains the value of the operand on the right



Operator	Name	Syntax	Operation
+=	Add then Assign	$\$x += \y	Simple Addition same as $\$x = \$x + \$y$
-=	Subtract then Assign	$\$x -= \y	Simple subtraction same as $\$x = \$x - \$y$
*=	Multiply then Assign	$\$x *= \y	Simple product same as $\$x = \$x * \$y$
/=	Divide then Assign (quotient)	$\$x /= \y	Simple division same as $\$x = \$x / \$y$
%=	Divide then Assign (remainder)	$\$x \% = \y	Simple division same as $\$x = \$x \% \$y$

Example: This example describes the assignment operator in PHP.

- PHP

```
<?php
```

```
// Simple assign operator
```

```
$y = 75;
```

```
echo $y, "\n";
```

```
// Add then assign operator
```

```
$y = 100;
```

```
$y += 200;
```

```
echo $y, "\n";
```



```
// Subtract then assign operator
```

```
$y = 70;
```

```
$y -= 10;
```

```
echo $y, "\n";
```

```
// Multiply then assign operator
```

```
$y = 30;
```

```
$y *= 20;
```

```
echo $y, "\n";
```

```
// Divide then assign(quotient) operator
```

```
$y = 100;
```

```
$y /= 5;
```

```
echo $y, "\n";
```

```
// Divide then assign(remainder) operator
```

```
$y = 50;
```

```
$y %= 5;
```

```
echo $y;
```

```
?>
```

Output:
75



300
60
600
20
0

Array Operators: These operators are used in the case of arrays. Here are the array operators along with their syntax and operations that PHP provides for the array operation.

Operator	Name	Syntax	Operation
+	Union	$\$x + \y	Union of both i.e., $\$x$ and $\$y$
==	Equality	$\$x == \y	Returns true if both has same key-value pair
!=	Inequality	$\$x != \y	Returns True if both are unequal
===	Identity	$\$x === \y	Returns True if both have the same key-value pair in the same order and of the same type
!==	Non-Identity	$\$x !== \y	Returns True if both are not identical to each other
<>	Inequality	$\$x <> \y	Returns True if both are unequal

Example: This example describes the array operation in PHP.

- PHP

```
<?php
```

```
$x = array("k" => "Car", "l" => "Bike");
```

```
$y = array("a" => "Train", "b" => "Plane");
```




```
var_dump($x + $y);

var_dump($x == $y) + "\n";

var_dump($x != $y) + "\n";

var_dump($x <> $y) + "\n";

var_dump($x === $y) + "\n";

var_dump($x !== $y) + "\n";

?>
```

Output:

```
array(4) {
  ["k"]=>
  string(3) "Car"
  ["l"]=>
  string(4) "Bike"
  ["a"]=>
  string(5) "Train"
  ["b"]=>
  string(5) "Plane"
}
bool(false)
bool(true)
bool(true)
bool(false)
bool(true)
```

Increment/Decrement Operators: These are called the unary operators as they work on single operands. These are used to increment or decrement values.



Operator	Name	Syntax	Operation
++	Pre-Increment	++\$x	First increments \$x by one, then return \$x
—	Pre-Decrement	—\$x	First decrements \$x by one, then return \$x
++	Post-Increment	\$x++	First returns \$x, then increment it by one
—	Post-Decrement	\$x—	First returns \$x, then decrement it by one

Example: This example describes the Increment/Decrement operators in PHP.

- PHP

```
<?php
```

```
$x = 2;
```

```
echo ++$x, " First increments then prints \n";
```

```
echo $x, "\n";
```

```
$x = 2;
```

```
echo $x++, " First prints then increments \n";
```

```
echo $x, "\n";
```

```
$x = 2;
```

```
echo --$x, " First decrements then prints \n";
```

```
echo $x, "\n";
```



```
$x = 2;
```

```
echo $x--, " First prints then decrements \n";
```

```
echo $x;
```

```
?>
```

Output:

3 First increments then prints

3

2 First prints then increments

3

1 First decrements then prints

1

2 First prints then decrements

1

String Operators:

This operator is used for the concatenation of 2 or more strings using the concatenation operator ('.'). We can also use the concatenating assignment operator ('.=') to append the argument on the right side to the argument on the left side.

Operator	Name	Syntax	Operation
.	Concatenation	$\$x.\y	Concatenated $\$x$ and $\$y$
.=	Concatenation and assignment	$\$x.=\y	First concatenates then assigns, same as $\$x = \$x.\$y$

Example: This example describes the string operator in PHP.

- PHP



```
<?php  
  
$x = "Geeks";  
  
$y = "for";  
  
$z = "Geeks!!!";  
  
echo $x . $y . $z, "\n";  
  
$x .= $y . $z;  
  
echo $x;  
  
?>
```

Output:

GeeksforGeeks!!!

GeeksforGeeks!!!

Spaceship Operators:

PHP 7 has introduced a new kind of operator called spaceship operator. The spaceship operator or combined comparison operator is denoted by “<=>”. These operators are used to compare values but instead of returning the boolean results, it returns integer values. If both the operands are equal, it returns 0. If the right operand is greater, it returns -1. If the left operand is greater, it returns 1. The following table shows how it works in detail:

Operator	Syntax	Operation
$\$x < \y	$\$x <=> \y	Identical to -1 (right is greater)
$\$x > \y	$\$x <=> \y	Identical to 1 (left is greater)
$\$x <= \y	$\$x <=> \y	Identical to -1 (right is greater) or identical to 0 (if both are equal)



Operator	Syntax	Operation
$\$x \geq \y	$\$x \leq \y	Identical to 1 (if left is greater) or identical to 0 (if both are equal)
$\$x == \y	$\$x \leq \y	Identical to 0 (both are equal)
$\$x != \y	$\$x \leq \y	Not Identical to 0

Example: This example illustrates the use of the spaceship operator in PHP.

- PHP

```
<?php
```

```
$x = 50;
```

```
$y = 50;
```

```
$z = 25;
```

```
echo $x <=> $y;
```

```
echo "\n";
```

```
echo $x <=> $z;
```

```
echo "\n";
```

```
echo $z <=> $y;
```

```
echo "\n";
```

```
// We can do the same for Strings
```



Insight Institute of Technology and Management, Nagpur

Approved by AICTE New Delhi and DTE Mumbai, Govt. of Maharashtra

Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University

```
$x = "Ram";  
  
$y = "Krishna";  
  
echo $x <=> $y;  
  
echo "\n";  
  
echo $x <=> $y;  
  
echo "\n";  
  
echo $y <=> $x;  
  
?>
```

Output:

```
0  
1  
-1  
1  
1  
-1
```



Flow-Control Statements:

If Else

PHP if else statement is used to test condition. There are various ways to use if statement in PHP.

- if
- if-else
- if-else-if
- nested if

PHP If Statement

PHP if statement allows conditional execution of code. It is executed if condition is true.

If statement is used to executes the block of code exist inside the if statement only if the specified condition is true.

Syntax

1. **if**(condition){
2. *//code to be executed*
3. }

Example

1. <?php
2. \$num=12;



```
3. if($num<100){
4.   echo "$num is less than 100";
5. }
6. ?>
```

Output:

```
12 is less than 100
```

PHP If-else Statement

PHP if-else statement is executed whether condition is true or false.

If-else statement is slightly different from if statement. It executes one block of code if the specified condition is **true** and another block of code if the condition is **false**.

Syntax

```
1. if(condition){
2.   //code to be executed if true
3. }else{
4.   //code to be executed if false
5. }
```

Example

```
1. <?php
2. $num=12;
3. if($num%2==0){
4.   echo "$num is even number";
5. }else{
6.   echo "$num is odd number";
7. }
8. ?>
```

Output:

```
12 is even number
```




PHP If-else-if Statement

The PHP if-else-if is a special statement used to combine multiple if?.else statements. So, we can check multiple conditions using this statement.

Syntax

1. **if** (condition1){
2. //code to be executed if condition1 is true
3. } **elseif** (condition2){
4. //code to be executed if condition2 is true
5. } **elseif** (condition3){
6. //code to be executed if condition3 is true
7.
8. } **else**{
9. //code to be executed if all given conditions are false
10. }

Example

1. <?php
2. \$marks=69;
3. **if** (\$marks<33){
4. echo "fail";
5. }
6. **else if** (\$marks>=34 && \$marks<50) {
7. echo "D grade";
8. }
9. **else if** (\$marks>=50 && \$marks<65) {
10. echo "C grade";
11. }
12. **else if** (\$marks>=65 && \$marks<80) {
13. echo "B grade";
14. }
15. **else if** (\$marks>=80 && \$marks<90) {
16. echo "A grade";



```
17. }  
18. else if ($marks>=90 && $marks<100) {  
19.     echo "A+ grade";  
20. }  
21. else {  
22.     echo "Invalid input";  
23. }  
24. ?>
```

Output:

B Grade

PHP nested if Statement

The nested if statement contains the if block inside another if block. The inner if statement executes only when specified condition in outer if statement is **true**.

Syntax

```
1. if (condition) {  
2.     //code to be executed if condition is true  
3. }  
4. if (condition) {  
5.     //code to be executed if condition is true  
6. }
```

Example

```
1. <?php  
2.     $age = 23;  
3.     $nationality = "Indian";  
4.     //applying conditions on nationality and age  
5.     if ($nationality == "Indian")  
6.     {  
7.         if ($age >= 18) {  
8.             echo "Eligible to give vote";
```



```
9.     }
10.    else {
11.        echo "Not eligible to give vote";
12.    }
13. }
14. ?>
```

Output:

Eligible to give vote

PHP Switch Example

```
1. <?php
2.     $a = 34; $b = 56; $c = 45;
3.     if ($a < $b) {
4.         if ($a < $c) {
5.             echo "$a is smaller than $b and $c";
6.         }
7.     }
8. ?>
```

Output:

34 is smaller than 56 and 45

Embedding PHP in Web Pages:

Although it is possible to write and run stand alone PHP programs, most PHP code is embedded in HTML or XML files. This is, after all, why it was created in the first place. The processing such documents involves replacing each chunk of PHP source code with the output it produces when executed. Because a single file contains PHP and non-PHP source code, we need a way to identify the regions of PHP code to be executed. PHP provides four different ways to do this:

- First, and preferred, method is XML.
- Second method is SGML.
- Third method is based on ASP tags.
- Fourth method uses the standard HTML



Embedding PHP in Web Pages- XML Style

XML Style: The way in which you normally embed PHP in a document is through what is commonly called a PHP statement, but is actually an XML directive. PHP is designed to be compatible with many standards and commonly used scripting approaches, making it easier to code. An XML processing directive is a command embedded in an XML document for some application to process. All XML processing directives take the form of:

```
<?appname information for application ?>
```

Notice that the statement opens and closes with nested brackets and question marks, and that the name of the application being referenced must come right after the first question mark, with no space before it. The information for the application can be a short statement, or, in the case of PHP, can go on for pages and pages.

PHP statement normally uses this form of inclusion in HTML documents. It looks like this:

```
<?php some phpstatements ?>
```

Line breaks are allowed within the XML processing directive, so the following is also legal.

```
<?php
```

```
Somephp statements
```

```
And some more statements
```

```
?>
```

If you want to write standards compliant code, or embed PHP in XML documents, you need to use this format. If you are not working with XML or don't need to adhere to standards, there are other approaches. Each may be appropriate within certain server situations. Since PHP is processed by the server, it can literally go anywhere in an HTML document in question. The code is processed before the Web browser sees the code, so the Web browser never sees the PHP. If working with XML, you need to make sure that any server-side XML processing is done after the PHP processing, or code the document with the knowledge that the PHP code cannot violate the syntax rules of the application you are working with. This means that XML documents containing PHP should still be well-formed and valid.

Embedding PHP in Web Pages- SGML Style

SGML Style: The "classic" style of embedding PHP comes from SGML instruction processing tags. To use this method, simply enclose the PHP in . Here's the "Hello world" example again:

```
<? echo "Hello, world"; ?>
```

This style, known as short tags, is the shortest and least intrusive, and it can be turned off so as to not clash with the XML PI (Process Instruction) tag in the php.ini initialization file. Consequently, if you want to write fully



portable PHP code that you are going to distribute to other people (who might have short tags turned off), you should use the longer style, which cannot be turned off. If you have no intention of distributing your code, you don't have an issue with telling people who want to use your code to turn on short tags, and you are not planning on mixing XML in with your PHP code, then using this tag style is okay.

Embedding PHP in Web Pages- ASP Style

ASP Style: Realizing that some old code editors might have problems with PHP, the authors of PHP included two additional ways to include PHP code in a document. One mimicks ASP tags. The other uses standard HTML `<script>` tags. The ASP tags are not enabled by default. To turn them on you need to find the asp tags flag in the php.ini file, or, if working from the source, build PHP with the `--enable-asp-tags` option.

It is not necessary for the server to actually process ASP tags as PHP tags, since the point of the compatibility with ASP tags is to allow users to work with editors that understand ASP tags but not PHP tags. When you are done with the document, you can just do a global search and replace to replace all ASP tags with PHP tags before moving the documents to production. The ASP tags take the following form:

`<% code goes here %>`

Like PHP tags, ASP tags can go anywhere in the document.

Embedding PHP in Web Pages- Script Style Script Style:

The final method of distinguishing PHP from HTML involves a tag invented to allow client-side scripting within HTML pages, the `<script>` tag. You might recognize it as the tag in which JavaScript is embedded. Since PHP is processed and removed from the file before it reaches the browser, you can use the `<script>` tag to surround PHP code. In order to use this method, simply specify "php" as the value of the language attribute of the tag:

`<script language="php"> echo "Hello, world"; </script>`

This method is most useful with HTML editors that work only on strictly legal HTML files and do not yet support XML processing commands.